SAGEVM

Assembly & Architecture

Version	v0.9.8
Architectures	SVM (Stack) & SRVM (Register)
Binary Format	SGVM (.sgvm)
Status	Active Development

A comprehensive guide to the low-level mechanics of SageVM. This document covers the dual-architecture model, the SGVM binary format, and provides detailed tutorials for writing and optimizing assembly for both stack and register-based backends.

At its core, a **Virtual Machine (VM)** is a software abstraction of a physical computer. It provides a consistent execution environment regardless of the underlying hardware. For SageVM, this abstraction is defined by two primary concepts:

- **Bytecode:** The instruction set of the VM. Unlike native machine code (x86_64, ARM), bytecode is portable and designed to be interpreted or JIT-compiled by the VM runtime.
- **Assembly:** The human-readable textual representation of bytecode. Assembly provides a 1:1 mapping to the VM's numeric opcodes, allowing developers to write low-level code without memorizing hex values.

Virtual Machines act as an intermediate layer in the compilation pipeline, decoupling the high-level language features from the complexities of silicon architectures.

There are two dominant architectural patterns for virtual machines, both of which are implemented within SageVM to serve different strategic needs.

2.1 Stack-based Virtual Machines (SVM)

A stack machine uses a **Last-In, First-Out (LIFO)** operand stack for all computations. To add two numbers, you "push" both onto the stack and then call "add", which pops them, computes the sum, and "pushes" the result back.

- **Pros:** Extremely compact bytecode (instructions don't need to specify register addresses); easy to generate from ASTs.
- **Cons:** Often requires more instructions than register machines; harder to optimize for modern CPUs without a sophisticated JIT.

2.2 Register-based Virtual Machines (SRVM)

A register machine operates on a **Virtual Register File**. Instructions explicitly state the source and destination registers (e.g., "add x10, x11, x12").

- **Pros:** Faster execution as it reduces stack traffic; closer mapping to physical hardware (like RISC-V or x86).
- **Cons:** Larger instruction size (as each instruction must carry register indices).

3. The SageVM Dual-Architecture Model

> SageVM is unique because it provides both architectures in a single unified toolchain. This allows developers to choose the best substrate for their specific task.

3.1 SVM (Standard Virtual Machine)

The SVM is the core execution layer of SageVM. It is optimized for portability and tight integration with the SageLang object system.

- Uses variable-length encoding (1-byte opcodes, 2-byte operands).
- Implements capability-based security at the opcode level.
- Primary target for initial compilation from SageLang source.

3.2 SRVM (Register Virtual Machine / RISC-V)

The SRVM is a high-performance backend that maps directly to the **RISC-V (RV64I)** specification. It is the foundation for the SageVM JIT and AOT pipelines.

- 32 x 64-bit general-purpose registers (x0 to x31).
- Fixed 32-bit instruction width for efficient decoding.
- Custom "VMSYS" extension for accessing SageVM-specific features like printing and object manipulation.

4. Binary Formats: SGVM vs. ELF

Understanding the container format is as important as the code inside it. SageVM uses a custom format called **SGVM**, which differs significantly from standard system formats like **ELF**.

4.1 What is ELF?

The **Executable and Linkable Format (ELF)** is the standard binary format for Linux and most Unix-like systems. It is designed to hold native machine code, symbol tables, and relocation information for physical hardware.

4.2 What is SGVM?

The **SGVM (.sgvm)** format is a portable container designed specifically for the Sage Virtual Machine. It contains:

- **Constant Pool:** A centralized table for strings and large numbers, reducing duplication.
- **Code Chunks:** Modular blocks of bytecode representing functions and scripts.
- **Capability Tags:** Metadata that restricts what the code is allowed to do (e.g., GPU access, Filesystem access).

4.3 Why not use ELF?

While ELF is powerful, SGVM provides **Architectural Neutrality**. An SGVM file compiled on an x86_64 machine will run identically on an ARM64 or RISC-V machine without modification. Furthermore, SGVM integrates directly with the SageLang Garbage Collector and Object System, which ELF cannot easily do.

5. Assembly Tutorials & Patterns

5.1 Basic Arithmetic & Stack Management (SVM)

```
; Compute (5 + 10) * 2
CONSTANT 5   ; Push 5
CONSTANT 10  ; Push 10
ADD          ; Stack now contains 15
CONSTANT 2   ; Push 2
MUL          ; Stack now contains 30
PRINT        ; Output top of stack
HALT         ; Terminate VM
```

5.2 Register-Based Logic (SRVM)

SRVM instructions use RISC-V mnemonics. Note the explicit register use.

```
; Register-based equivalent
ldc x10, 5        ; Load constant index 5 into x10
ldc x11, 10       ; Load constant index 10 into x11
add x10, x10, x11 ; x10 = x10 + x11
li  x11, 2        ; Load immediate 2 into x11
mul x10, x10, x11 ; x10 = x10 * x11
vmsys x10, 0x09   ; VM_PRINT register x10
vmsys x0, 0x01    ; VM_HALT
```

5.3 Control Flow & Loops

Loops in SVM use relative jumps, while SRVM uses labels and branch instructions.

```
; SVM Loop: Print 1 to 5
CONSTANT 1
DEFINE_GLOBAL "i"
LOOP:
  GET_GLOBAL "i"
  CONSTANT 5
  LESS_EQUAL
  JUMP_IF_FALSE EXIT
  GET_GLOBAL "i"
  PRINT
  GET_GLOBAL "i"
  CONSTANT 1
  ADD
  SET_GLOBAL "i"
  JUMP LOOP
EXIT:
  HALT
```

6. Advanced System Engineering

6.1 Exception Handling Architecture

SageVM uses an explicit handler stack. This ensures that even in low-level assembly, memory safety and resource cleanup are guaranteed.

```
SETUP_TRY CATCH_BLOCK ; Register a handler at CATCH_BLOCK
CONSTANT "Dangerous Operation"
RAISE                 ; Trigger exception
END_TRY               ; Never reached
CATCH_BLOCK:
  PRINT               ; Exception value is on stack
  HALT
```

6.2 GPU Interfacing via Assembly

SageVM provides 28 dedicated GPU opcodes. These allow you to build Vulkan command buffers directly in assembly.

```
OP_GPU_POLL_EVENTS
CONSTANT 0; OP_GPU_ACQUIRE_IMG
CONSTANT 1; OP_GPU_BEGIN_COMMANDS
OP_GPU_CMD_BEGIN_RP
OP_GPU_CMD_DRAW
OP_GPU_CMD_BIND_GP
OP_GPU_CMD_END_RP
OP_GPU_END_COMMANDS
OP_GPU_PRESENT
```

End of Document · SageVM Assembly & Architecture Guide v0.9.8 · Night-Traders-Dev